

Rapport final de projet — MediPlan

Plateforme web de gestion des rendez-vous médicaux

Cours	Projet intégrateur 030747 — Collège La Cité
Session	Printemps 2026
Équipe	Souleymane DIALLO · Zakaria Lahouiri · Larbi Saib
Date	12 août 2026
Application	ca-mediplan-frontend.ashytree-9ad5012f.canadacentral.azurecontainerapps.io
Dépôt	github.com/soultaka19/mediplan

1. L'idée du projet

MediPlan est un agenda partagé pour une clinique médicale de petite taille. Il réunit dans un seul écran ce qui est aujourd'hui éparpillé entre un téléphone, un cahier et un fichier partagé : les disponibilités des médecins, les rendez-vous, l'état de la journée en cours et les chiffres d'activité.

L'idée tient en une phrase : outiller la réception, pas remplacer le téléphone.

2. Le problème identifié

Nous sommes partis de cinq irritants concrets d'une clinique de proximité :

- deux personnes réservent le même créneau, et il faut le démêler devant le patient ;
- un rendez-vous annulé n'est jamais remis en circulation, donc le créneau est perdu ;
- personne n'a de vue partagée sur la journée : on se lève pour demander qui est arrivé ;
- le patient doit se créer un compte pour exister, alors qu'au téléphone il n'en ouvre pas ;
- rien n'est mesuré, donc impossible de savoir si l'agenda se remplit ou si les absences dérapent.

Un constat a réorienté le projet. Notre cahier des charges initial plaçait le patient au centre, sur le modèle de Doctolib. En travaillant le besoin, nous avons retenu que dans une petite clinique, le patient appelle — il ne s'inscrit pas. L'utilisateur qui vit le problème est la réception.

Ce déplacement est le choix produit le plus important du projet. Il a changé le modèle de données, l'ordre des écrans et le fil de la démonstration.

Les utilisateurs visés sont donc, dans l'ordre : l'administrateur de clinique (la réception), le médecin, le patient, et le super administrateur. Chacun ne voit que les données de sa clinique, et cette règle est appliquée par le serveur.

3. La solution développée

Le médecin publie une plage de disponibilité : une date, deux bornes horaires, une durée de créneau. Le système génère seul les créneaux réservables. Une plage de 9 h à 12 h en créneaux de 30 minutes en produit six, sans autre saisie.

La réception réserve pour un patient au téléphone en trois gestes : le médecin, le créneau, le patient. Le patient est créé au comptoir comme *patient léger* : il existe dans le système, sans compte ni mot de passe.

Un patient inscrit peut aussi réserver seul. Les deux canaux passent par la même transaction et le même index en base, donc la garantie ne dépend pas du canal utilisé.

La journée se suit dans le flux du jour, vue partagée entre la réception et le médecin. Chaque rendez-vous parcourt : Réservé Arrivé En consultation Terminé, ou Annulé.

L'annulation exige un motif : le bouton de confirmation reste inactif tant que le champ est vide. Le créneau libéré redevient immédiatement reservable.

Les statistiques donnent le volume de rendez-vous, le taux d'absence et le taux d'occupation, par période et par médecin.

Le choix technique dont nous sommes le plus fiers

Empêcher qu'un créneau soit pris deux fois paraît simple : on vérifie qu'il est libre, puis on écrit. Cette approche ne peut pas marcher. Entre la vérification et l'écriture, une autre requête peut passer : les deux voient le créneau libre, les deux réservent. Le défaut n'apparaît que sous charge, donc jamais pendant les tests manuels, et toujours le jour où deux personnes travaillent en même temps.

Nous avons sorti la garantie du code pour la poser dans la base, sous forme de contrainte d'unicité. Ce n'est plus notre code qui arbitre, c'est PostgreSQL.

La contrainte a ensuite été transformée en index unique **partiel**, qui exclut les rendez-vous annulés. C'est ce détail qui rend l'annulation possible : sans lui, un créneau annulé resterait bloqué à jamais.

La leçon : quand une garantie dépend de la discipline du développeur, elle finit par tomber. Posée dans la base, elle tient même si une fonctionnalité future oublie de vérifier.

L'architecture

Trois tiers : un frontend Angular servi par nginx, une API NestJS, une base PostgreSQL. En développement, les trois tournent sous Docker Compose. En ligne, le frontend et l'API sont sur Azure Container Apps et la base est infogérée chez Neon.

Le backend n'a **aucune adresse publique** : il n'est joignable que par le frontend. Cela le met hors d'atteinte depuis Internet et supprime tout besoin de CORS, puisqu'il n'y a jamais de requête inter-origines.

Toute l'infrastructure est décrite en Bicep : aucune ressource n'a été créée à la main. Le coût est maintenu à environ 0 \$ par mois, contrainte d'un crédit étudiant non renouvelable. La contrepartie est un démarrage à froid de 10 à 15 secondes.

4. Les étapes du travail

Date	Jalon
28 mai 2026	Cahier des charges
3 juin	Dossier de conception : 7 cas d'utilisation, classes, séquence, ERD
17 juin	Socle technique et authentification complets
8 juillet	Patient léger, disponibilités, prise de rendez-vous
21-22 juillet	Refonte UX/UI et KPI réels du tableau de bord
29 juillet	Mise en ligne sur Azure
10 août	Statistiques, export CSV, notifications
11 août	Réservation patient en libre-service
13 août	Présentation finale

Ces étapes se sont chevauchées. Nous avons découpé le travail **par tranche verticale** : chacun mène sa fonctionnalité de la base de données jusqu'à l'écran, plutôt qu'une séparation « un front / un back ». La mise en ligne s'est faite pendant le dernier sprint et non à la fin, ce qui nous a permis de découvrir tôt les écarts entre le local et la production.

L'organisation. Jira, projet MEDIPLAN : 51 tickets, 7 épiques. GitHub : 21 pull requests, 3 contributeurs sur main. Le flux est un ticket, une branche, une pull request, une revue. L'intégration continue bloque la fusion si la compilation ou les tests échouent.

5. Ce qui a changé pendant le projet

C'est la partie la plus utile de ce rapport, parce que c'est là que nous avons appris quelque chose.

Le patient est passé au second plan

Prévu : le patient réserve en ligne. Réalisé : la réception réserve pour lui, et il peut aussi réserver seul. Le modèle du patient léger est né de ce changement. Le canal libre-service a été livré tard, en complément, et non comme fondation.

Le déploiement local est devenu un déploiement réel

Le cahier des charges écrivait : « aucun hébergement cloud requis pour la validation ». Nous avons mis l'application en ligne quand même, parce qu'une application qui ne tourne que sur nos machines ne prouve pas grand-chose. La mise en ligne a immédiatement révélé trois défauts invisibles en local (§ 7).

Les disponibilités récurrentes ont été abandonnées

Prévu : des plages récurrentes avec exceptions. Réalisé : des plages **datées**, plus les congés. C'était un risque identifié dès le départ dans notre matrice, et il s'est réalisé. Nous l'avons traité en réduisant le périmètre plutôt qu'en repoussant la livraison.

Notre définition de « terminé » a dû être réécrite

C'est le changement le plus important, et il vient d'un échec.

Des branches sont restées non fusionnées pendant des semaines, jusqu'à **56 commits de retard** sur main, dont une refonte complète de l'interface. Le code était bon mais ne s'appliquait plus sur rien de reconnaissable. Surtout : **cinq tickets étaient marqués « Terminé » dans Jira sans exister dans le produit.**

Nous avons corrigé en trois temps. D'abord la réintégration branche par branche, en pull requests revues, conflit par conflit, avec une règle simple : conserver les deux côtés, parce que ces conflits n'étaient pas des désaccords mais deux ajouts au même endroit. Ensuite, deux branches n'ont pas été reprises — à trois jours de l'échéance, les rejouer par-dessus l'interface refondue risquait de casser le cœur du produit ; les tickets sont repassés « À faire » avec la raison écrite. Enfin, la définition de « terminé » a été resserrée : **fusionné dans `main`, CI verte.**

La leçon : un ticket n'est pas terminé quand le code est écrit, mais quand il est fusionné. Une branche qui vit trois semaines coûte plus cher à intégrer qu'elle n'a coûté à écrire.

Un **Could** a été livré pendant qu'un **Must** ne l'était pas

Notre priorisation classait l'export CSV en **Could** et la gestion des médecins en **Must**. L'export est livré, la gestion des médecins non. L'explication est simple et peu flatteuse : l'export était court et bien défini, la gestion des médecins était longue et floue. Nous avons suivi le plus facile au lieu du plus important, et nous n'avons pas relu notre priorisation une seule fois en cours de projet.

Notre rythme a été irrégulier

En relisant l'historique du dépôt, deux semaines comptent deux commits et deux autres en comptent trente-neuf. Nous avons produit à l'approche des échéances plutôt que régulièrement — et c'est la même cause qui produit le problème d'intégration ci-dessus : on ne fusionne pas ce sur quoi on ne travaille pas cette semaine-là.

6. L'état actuel de la solution

L'application est **en ligne, fonctionnelle et démontrable de bout en bout**. Vérifié le 12 août 2026 : sonde de disponibilité en 0,44 seconde, connexion en 0,99 seconde, premier accès à froid en 9,9 secondes.

Neuf fonctionnalités livrées : authentification complète ; RBAC à 4 rôles borné par clinique ; disponibilités et génération automatique des créneaux ; réservation par la réception avec patient léger ; réservation en libre-service par le patient ; flux du jour et transitions de statut ; annulation avec motif et libération du créneau ; notifications internes ; statistiques et export CSV.

Par rapport au cahier des charges initial : sur 11 fonctionnalités, 7 livrées, 3 partielles, 1 absente. Sur 11 livrables, 9 produits. Deux éléments ont été livrés en plus de ce qui était demandé : la mise en ligne réelle et le second canal de réservation.

7. Tests réalisés et résultats

Ce que nous avons choisi de tester

Nous n'avons pas visé une couverture uniforme. À trois sur un semestre, elle aurait consommé le temps du développement sans rien garantir de plus. Nous avons testé ce qui casse sans prévenir : les règles métier (une transition de statut invalide ou un créneau pris deux fois corrompt des données sans message d'erreur), la sécurité (un défaut de contrôle d'accès ne se voit jamais à l'usage), et le comportement des formulaires. L'apparence est hors périmètre.

Les résultats

203 tests automatisés, tous verts au 12 août 2026 : 70 côté backend en 11 suites, 133 côté frontend en 28 suites.

Le parcours complet est validé **manuellement** avant chaque démonstration, selon un scénario écrit en le jouant sur l'application en ligne. Le passage du 10 août a vérifié 14 points : génération des créneaux, retrait du créneau réservé, cycle de vie complet, motif obligatoire à l'annulation, créneau effectivement libéré, notifications émises, RBAC visible à l'écran, export CSV, statistiques sur données réelles. Aucune erreur dans la console du navigateur.

La garantie anti-double-réservation a été vérifiée en concurrence réelle sur l'application déployée : deux réservations simultanées sur le même créneau, une requête en 201 et l'autre en 409.

Les bogues qui nous ont le plus appris

Le 404 invisible en local. Une fois en ligne, tous les appels à l'API renvoyaient 404, alors que tout fonctionnait en local. Nous avons cherché dans le code, dans les routes, dans la configuration, sans résultat — le problème n'y était pas. Le déblocage est venu d'un changement de méthode : au lieu de chercher ce qui était cassé, chercher ce qui différait entre les deux environnements. Notre proxy transmettait l'en-tête Host demandé par le navigateur ; Azure Container Apps route selon cet en-tête, alors que Docker Compose ne route pas ainsi.

Les fins de ligne Windows. Un fichier de démarrage récupéré en CRLF empêchait un conteneur de démarrer.

Le fuseau horaire. Les horaires de la clinique apparaissaient décalés de quatre heures, parce que le conteneur tourne en UTC.

Aucun de ces trois n'était un bogue de code : tous venaient d'un écart entre deux environnements. Un environnement de développement qui fonctionne ne prouve rien sur la production. Les deux ne diffèrent pas seulement par leurs données, mais par leur façon d'acheminer une requête, de lire un fichier et de compter les heures.

Ce qui n'a pas été testé

Aucun test de bout en bout automatisé, aucun test de charge, aucun audit Lighthouse. La couverture n'a pas été mesurée, alors que notre cahier des charges visait 70 % sur les modules métier. Sur les sept indicateurs chiffrés que nous avons fixés en mai, **aucun n'a été mesuré selon la méthode annoncée.**

8. Les limites de la solution

Fonctionnalités absentes. Modifier ou déplacer un rendez-vous (il faut annuler puis réserver à nouveau). Annulation par le patient — elle suppose une règle de délai minimum que nous n'avons pas écrite, et nous préférons ne pas livrer l'une sans l'autre. Gestion des médecins et configuration de clinique par l'interface, l'accès rapide correspondant étant visible mais grisé. Décalage en bloc des rendez-vous, codé et testé mais non intégré. Aucune notification vers l'extérieur, ni courriel ni SMS.

Limites techniques. Le multi-clinique est dans le modèle et appliqué côté sécurité, mais aucun écran ne le pilote : MediPlan vise **une clinique fonctionnelle**. Pas de pagination serveur — c'est le premier mur qu'une clinique à fort volume rencontrerait. La sécurité est incomplète sur les couches secondaires : pas de jeton de rafraîchissement, **aucun en-tête HTTP de sécurité**, aucune journalisation d'audit. Le socle tient (bcrypt, validation stricte, contrôle d'accès appliqué, aucun secret versionné), mais la défense en profondeur manque. Dette de formatage assumée : 233 fichiers jamais passés au formateur et 9 avertissements d'accessibilité.

Limites de méthode. Notre définition de « terminé » a été trop souple pendant une partie du projet. Les revues de code sont arrivées tard. Le volume de commits est très inégal entre les membres.

9. La contribution de chaque membre

Les attributions ci-dessous ont été retracées commit par commit. Le détail complet est dans docs/presentation/CONTRIBUTIONS.md.

	Souleymane DIALLO	Zakaria Lahouiri	Larbi Saib
Domaine	Socle, sécurité, mise en ligne, espace patient	Le temps du médecin	Le rendez-vous et sa mesure

Conception	Cahier des charges, 7 cas d'utilisation, ERD	Diagramme de classes	Diagrammes de séquence
Épisodes Jira	E1, E2, E4, E7	E3, E5, E6	—
Commits sur main	72	8	4

Le volume de commits est très inégal et nous l'assumons. Souleymane a porté le socle et l'intégration, ce qui produit mécaniquement beaucoup de commits, tandis que Zakaria et Larbi ont livré des fonctionnalités complètes en peu de commits. Ce chiffre mesure une façon de travailler, pas une contribution.

Souleymane DIALLO a réalisé le cahier des charges et les cas d'utilisation, le monorepo, Docker et l'intégration continue, l'authentification JWT avec bcrypt, verrouillage et réinitialisation, le contrôle d'accès à 4 rôles, les écrans d'authentification, le design system et la refonte de l'interface, le passage de la contrainte d'unicité à un index **partiel** qui rend l'annulation possible, l'annulation avec motif, la réservation en libre-service, la mise en ligne Azure et la réintégration finale des branches. **Sa difficulté** : le 404 qui n'existait qu'en production. **Sa solution** : cesser de chercher ce qui était cassé pour chercher ce qui différait entre les environnements.

Zakaria Lahouiri a réalisé le diagramme de classes, les disponibilités par plages datées avec génération automatique des créneaux, les plages de congé, le flux clinique du jour et ses transitions de statut, le module complet de notifications internes avec émission automatique sur les trois événements du cycle de vie, et l'export CSV borné à la clinique. **Sa difficulté** : ses branches avaient pris jusqu'à 56 commits de retard sur une interface entre-temps refondue. **Sa solution** : rebasage et réintégration par pull requests, conflit par conflit, en conservant les deux apports.

Larbi Saib a réalisé les diagrammes de séquence, le modèle du patient léger, le socle technique du rendez-vous (entités, migration, service, contrôleur, validation), la contrainte d'unicité anti-double-réservation posée dès la première version, et le module d'agrégation avec l'écran de statistiques qui a remplacé les compteurs factices du tableau de bord par des chiffres réels. **Sa difficulté** : garantir qu'un créneau ne soit jamais réservé deux fois. **Sa solution** : comprendre pourquoi « vérifier puis écrire » ne peut pas marcher, et poser la garantie dans la base.

En commun : la décision de découper par tranche verticale, la révision de la définition de « terminé », la réflexion UX/UI et la préparation de la présentation.

10. Améliorations et prochaines étapes

L'ordre n'est pas arbitraire : finir ce qui est écrit, sécuriser ce qui existe, puis seulement ajouter. Ajouter sur une base fragile est ce qui nous a coûté le plus cher.

D'abord solder l'existant. Intégrer le décalage en bloc, déjà écrit et testé. Livrer la configuration de clinique et la gestion des médecins, les deux seules exigences **Must** encore ouvertes. Passer le formateur sur tout le dépôt.

Puis fiabiliser. Poser les en-têtes HTTP de sécurité et la journalisation d'audit : c'est notre plus grand écart pour le plus petit effort, moins d'une journée. Ajouter un jeton de rafraîchissement avec rotation. Écrire les tests de bout en bout. Ajouter la pagination serveur. Rendre le lint bloquant une fois la dette résorbée.

Ensuite seulement, étendre la valeur. Les rappels par courriel ou SMS d'abord : c'est la fonctionnalité qui attaque directement le taux d'absence, le seul chiffre que la clinique voit sur sa facture. Puis une liste d'attente qui proposerait automatiquement un créneau libéré par une annulation. Puis l'annulation par le patient avec son délai minimum. Et enfin l'ouverture à un réseau de cliniques, qui demanderait de construire l'interface d'administration aujourd'hui absente.

11. Conclusion

MediPlan est livré : une application déployée, testée et démontrable de bout en bout. La double réservation est devenue techniquement impossible, la journée de clinique tient dans un écran partagé, et un créneau annulé repart à la réservation.

Le produit est en deçà de notre cahier des charges initial sur trois points : la gestion des médecins et des cliniques par l'interface, la modification d'un rendez-vous, et la validation automatisée de bout en bout. Il le dépasse sur deux autres : la mise en ligne réelle, jamais exigée, et le second canal de réservation.

La cause des manques n'est pas technique, elle est de méthode. Nous avons travaillé longtemps sans intégrer, et notre définition de « terminé » a laissé du code exister sans être dans le produit. Le correctif est appliqué, et c'est ce que nous retenons le plus de ce projet.

Où trouver quoi

Élément	Emplacement
Code source	apps/backend/, apps/frontend/
Cahier des charges final	docs/cahier-des-charges/
Dossier de conception	docs/conception/
Manuel d'utilisation	docs/guide-utilisation/
Vidéo démonstrative	MediPlan-Demo.mp4, à la racine du dépôt
Tests, résultats, bogues	docs/tests/plan-et-resultats.md
Contributions individuelles	docs/presentation/CONTRIBUTIONS.md
Instructions de lancement	README.md
Suivi de projet	Jira, projet MEDIPLAN